



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)
DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

LEIM

LICENCIATURA EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
UNIDADE CURRICULAR DE PROJETO

Sistema para geração automática de Certificados de Curso



Nilo Duarte 48155

Emiliana Prates 51635

Orientador(es)

Professor [Doutor] André Mendes

Maio, 2026

Resumo

A emissão de Certificados de Conclusão de Curso é uma tarefa da responsabilidade dos Serviços Académicos das instituições de ensino que, do ponto de vista administrativo, se caracteriza por ser um processo demoroso e propenso a erros, em particular em cursos com um elevado número de alunos, onde a emissão é tipicamente feita de forma manual e individualizada. A automatização deste processo constitui, por isso, uma necessidade real e urgente para a melhoria da eficiência dos serviços administrativos.

No âmbito deste projeto, foi desenvolvido um sistema *web* para a geração automática de Certificados de Curso, destinado ao pessoal administrativo de instituições de ensino e centros de formação. O sistema foi concebido com base numa arquitetura *Model-View-Controller* (MVC) e implementado em TypeScript com recurso ao *framework* Express.js, adotando padrões de desenvolvimento que promovem a modularidade, a manutenibilidade e a extensibilidade da aplicação.

As funcionalidades centrais desenvolvidas incluem: a geração automática de certificados em formato PDF a partir de templates personalizáveis, concebidos pelo administrador através de uma interface gráfica intuitiva; a assinatura digital dos documentos gerados, implementada segundo a norma PKCS#7, que garante a autenticidade e integridade dos certificados emitidos; e a distribuição automática dos certificados aos alunos por correio eletrónico, com o ficheiro PDF incluído como anexo, utilizando o protocolo SMTP através da biblioteca Nodemailer. O controlo de acessos é assegurado pelo modelo RBAC (*Role-Based Access Control*), implementado com recurso à biblioteca Casbin, que distingue dois perfis de utilização dentro dos serviços administrativos: o perfil de administrador, com acesso total à gestão de utilizadores, templates e emissão de certificados, e o perfil de utilizador regular, com acesso mais restrito.

A validação do sistema foi conduzida através de testes funcionais e de integração que cobriram os principais fluxos operacionais: a geração de certificados em lote com processamento segmentado para garantir a estabilidade dos recursos do servidor, a verificação da integridade das assinaturas digi-

tais em leitores PDF externos (Adobe Acrobat Reader) e o comportamento resiliente do sistema perante falhas nas configurações do serviço de correio eletrônico.

Os resultados obtidos demonstram que o sistema desenvolvido cumpre os objetivos propostos, constituindo uma alternativa *open-source*, auto-hospedada e totalmente configurável às soluções comerciais existentes, as quais implicam custos recorrentes e dependência de infraestruturas externas.

Abstract

The issuance of Course Completion Certificates is a task under the responsibility of the Academic Services of educational institutions which, from an administrative standpoint, is characterised as a time-consuming and error-prone process, particularly in courses with a large number of students where certificates are typically issued manually and individually. The automation of this process therefore represents a genuine and pressing need for improving the efficiency of administrative services.

In the scope of this project, a web system for the automatic generation of Course Certificates was developed, aimed at the administrative staff of educational institutions and training centres. The system was designed around a Model-View-Controller (MVC) architecture and implemented in TypeScript using the Express.js framework, following development patterns that promote modularity, maintainability, and extensibility.

The core functionalities developed include: the automatic generation of certificates in PDF format from administrator-defined customisable templates, managed through an intuitive graphical interface; the digital signing of generated documents, implemented according to the PKCS#7 standard, ensuring the authenticity and integrity of the issued certificates; and the automatic distribution of certificates to students by email, with the PDF file included as an attachment, using the SMTP protocol via the Nodemailer library. Access control is enforced through the Role-Based Access Control (RBAC) model, implemented using the Casbin library, which distinguishes two administrative profiles: the administrator role, with full access to user management, templates and certificate issuance, and the regular user role, with more restricted access.

System validation was carried out through functional and integration tests covering the main operational workflows: batch certificate generation with segmented processing to ensure server resource stability, verification of digital signature integrity using external PDF readers (Adobe Acrobat Reader), and the system's resilient behaviour in the face of email service configuration failures.

The results obtained demonstrate that the developed system meets its proposed objectives, providing an open-source, self-hosted, and fully configurable alternative to existing commercial solutions, which typically involve recurring costs and dependency on external infrastructure.

Agradecimentos

Queremos agradecer ao nosso orientador André Mendes, pelo apoio fornecido na realização deste projeto e também ao Instituto Superior de Engenharia de Lisboa (ISEL) pela formação que nos foi dada nos últimos anos. ...

Índice

Resumo	i
Abstract	iii
Agradecimentos	v
Índice	vii
Lista de Tabelas	xi
Lista de Figuras	xiii
1 Introdução	1
1.1 Enquadramento e Motivação	1
1.2 Ideias Essenciais e Valor Acrescentado	2
1.3 Principais Contribuições	2
1.4 Processo de Desenvolvimento e Abordagem	3
1.5 Validação e Apreciação Global	3
1.6 Estrutura do Relatório	3
2 Trabalho Relacionado	5
2.1 Processo de Certificação de Formação em Portugal	5
2.2 Soluções Comerciais Existentes	6
2.3 Geração Programática de Documentos PDF	8
2.4 Assinatura Digital de Documentos	8
2.5 Distribuição por Correio Eletrónico Automatizado	9
2.6 Síntese Comparativa	10
2.6.1 Delimitação do Contexto	10

2.6.2	Aspetos Diferenciadores e Inovadores	11
2.6.3	Pressupostos Teóricos e Tecnológicos	11
3	Modelo Proposto	13
3.1	Requisitos	13
3.1.1	Atores do Sistema	13
3.1.2	Requisitos Funcionais e Não-Funcionais	14
3.1.3	Casos de Utilização	15
3.2	Fundamentos	21
3.2.1	Arquitetura Model-View-Controller (MVC)	21
3.2.2	Motor de Renderização de Documentos PDF	23
3.2.3	Segurança e Controlo de Acessos	23
3.3	Abordagem	23
3.3.1	Estruturação e Conceção do Layout de Templates	24
3.3.2	Algoritmo de Geração de Certificados em Lote	24
3.3.3	Fluxo de Distribuição por Correio Eletrónico	25
4	Implementação do Modelo	27
4.1	Arquitetura Geral do Sistema	27
4.1.1	Camada de Modelo	28
4.1.2	Camada de Controlo	29
4.1.3	Camada de Visualização	29
4.1.4	Diagrama de Componentes do Sistema	29
4.2	Geração de Certificados PDF	30
4.3	Assinatura Digital	31
4.4	Envio Automático de Emails	31
4.5	Segurança e Proteção de Dados	32
4.6	Controlo de Acessos baseado em RBAC	32
4.7	Fluxo de Trabalho Integrado	33
5	Validação e Testes	35
5.1	Validação da Geração de Certificados em Lote	35
5.2	Validação da Assinatura Digital	36
5.3	Testes de Integração com o Serviço de Email	36
5.4	Validação do Controlo de Acessos (RBAC)	37
5.5	Conclusão da Validação	37

6	Conclusões e Trabalho Futuro	39
6.1	Conclusões	39
6.2	Trabalho Futuro	41
A	Gestão e Controlo de Versões	43
B	Estrutura de Layout JSON dos Certificados	45
C	Ficheiros com etiqueta #my_code	47
	Bibliografia	49

Lista de Tabelas

2.1	Comparação entre soluções de emissão de certificados digitais .	10
3.1	CU1 — Criar chave de acesso para funcionário	17
3.2	CU2 — Autenticar funcionário na plataforma	18
3.3	CU3 — Obter dados dos alunos aprovados	19
3.4	CU4 — Gerar e assinar digitalmente o certificado	20
3.5	CU5 — Notificar e disponibilizar o certificado ao aluno	21
5.1	Matriz de validação de acessos (RBAC)	37

Lista de Figuras

3.1	Funções e Atributos do Sistema	15
3.2	Diagrama de Casos de Utilização	16
4.1	Arquitetura em camadas do sistema (MVC + Service + DAO)	28
4.2	Diagrama de componentes do sistema	29
4.3	Diagrama de atividade do cenário principal (workflow geral) .	34

Capítulo 1

Introdução

Este capítulo serve como introdução geral ao relatório, descrevendo o enquadramento do projeto desenvolvido, a sua motivação, o valor acrescentado face a soluções comerciais existentes, os principais contributos obtidos e a estrutura geral das secções do documento.

Este relatório descreve o desenvolvimento do Projeto n.º 41 - Sistema para Geração Automática de Certificados de Curso, realizado no âmbito da Licenciatura em Engenharia Informática e Multimédia. O objetivo principal do projeto consiste no desenho e implementação de uma plataforma informática que simplifique e desmaterialize o processo de emissão e envio de certificados aos estudantes, prestando suporte à atividade do pessoal administrativo.

1.1 Enquadramento e Motivação

A emissão de Certificados de Conclusão de Curso é um procedimento tipicamente a cargo dos Serviços Académicos ou de secretarias de formação de instituições de ensino. Do ponto de vista administrativo, este caracteriza-se por ser um processo tradicionalmente manual, em que cada certificado é preenchido e processado individualmente por aluno. Esta abordagem traduz-se num esforço administrativo dispendioso e moroso, especialmente em turmas ou cursos com elevada afluência de formandos, estando adicionalmente sujeito a erros de transcrição de dados e a falhas humanas no envio dos documentos.

Deste modo, a melhoria de eficiência nas secretarias e a otimização dos fluxos de trabalho dependem de soluções de automação capazes de assegurar a integridade dos dados e agilizar os tempos de processamento da instituição.

1.2 Ideias Essenciais e Valor Acrescentado

O sistema proposto assenta no desenvolvimento de uma aplicação *web* integrada que permite a criação visual de modelos (*templates*) de certificados, a injeção automática de dados dos alunos nestes modelos e a distribuição imediata dos ficheiros resultantes.

A principal distinção e valor acrescentado do sistema desenvolvido face às soluções comerciais existentes no mercado (como a Accredible ou a Certifier) reside no facto de se tratar de uma solução de código aberto (*open-source*), concebida para ser instalada e executada de forma local nos servidores da própria instituição de ensino (*self-hosted*). Isto traduz-se numa total autonomia na gestão de dados sensíveis dos formandos, na eliminação de custos recorrentes de subscrição de serviços terceiros e na flexibilidade total de personalização dos fluxos e do aspeto visual dos certificados.

1.3 Principais Contribuições

No decurso deste projeto, foram alcançados os seguintes marcos e contributos técnicos:

- **Geração Programática em Lote:** Um motor baseado na biblioteca *Puppeteer* que interpreta e renderiza especificações modernas de HTML e CSS, convertendo-as em ficheiros PDF com alta fidelidade visual. O processo foi otimizado para gerar certificados em lotes contidos (segmentos de 5 alunos), garantindo a estabilidade de recursos do servidor e mitigando falhas de falta de memória (*Out Of Memory*).
- **Assinatura Digital Integrada:** Integração da biblioteca *node-signpdf* para aplicação automática de assinaturas criptográficas baseadas na norma PKCS#7 com certificados *.p12*. Deste modo, confere-se validade legal e garantia de integridade física a cada diploma exportado.
- **Expedição de Notificações:** Desenvolvimento de um módulo de distribuição baseado em *Nodemailer* (via protocolo SMTP), que anexa e envia automaticamente o certificado assinado para o correio eletrónico de cada formando.

- **Gestão de Acessos Segura:** Implementação de controlo de acessos baseado em funções (RBAC) com a biblioteca *Casbin*, separando claramente as operações executadas pelo administrador e por utilizadores administrativos comuns na plataforma.

1.4 Processo de Desenvolvimento e Abordagem

Para o desenvolvimento do software, foi adotado o ecossistema Node.js com a linguagem TypeScript, tirando partido da tipagem estática para aumentar a robustez do código. O servidor *web* foi construído sobre o *framework* Express.js, adotando a arquitetura *Model-View-Controller* (MVC). Esta arquitetura foi estendida através de camadas de Serviço (*Service*) e de Acesso a Dados (*Data Access Object* — DAO), o que permitiu isolar completamente a lógica de negócio do armazenamento persistente de dados, facilitando a portabilidade de dados e a realização de testes.

1.5 Validação e Apreciação Global

Os testes e a validação do sistema confirmaram o cumprimento rigoroso dos requisitos estabelecidos. O motor de geração revelou-se estável sob carga elevada de processamento, a validade e inviolabilidade da assinatura digital aplicada foram verificadas e autenticadas com sucesso por leitores de PDF de referência de mercado (como o *Adobe Acrobat Reader*) e a resiliência a falhas de rede e configuração de correio eletrónico foi devidamente assegurada. Em suma, o sistema constitui uma ferramenta pronta para produção e capaz de responder à desmaterialização burocrática institucional.

1.6 Estrutura do Relatório

O presente documento encontra-se estruturado em seis capítulos principais, organizados da seguinte forma:

- O **Capítulo 1 — Introdução** contextualiza o projeto, apresentando a motivação, contribuições e a estrutura de leitura do relatório.

- O **Capítulo 2 — Trabalho Relacionado** efetua uma análise do estado da arte, revendo soluções comerciais de mercado e tecnologias para geração de PDF, assinaturas digitais, SMTP e controlo de acessos, concluindo com uma síntese comparativa.
- O **Capítulo 3 — Modelo Proposto** detalha a especificação dos requisitos funcionais e não-funcionais, a modelação de casos de utilização e os pilares de arquitetura conceptual do sistema.
- O **Capítulo 4 — Implementação do Modelo** descreve a arquitetura lógica e física construída, os algoritmos e fluxos detalhados e as opções técnicas implementadas no código.
- O **Capítulo 5 — Validação e Testes** expõe as estratégias e metodologias utilizadas para verificar o comportamento do sistema, detalhando os testes funcionais e de integração efetuados.
- O **Capítulo 6 — Conclusões e Trabalho Futuro** sintetiza os objetivos atingidos, a apreciação do trabalho realizado e as direções propostas para desenvolvimento futuro da plataforma.

Capítulo 2

Trabalho Relacionado

Neste capítulo apresenta-se o enquadramento do processo global de emissão de certificados em Portugal, as principais soluções comerciais existentes no mercado, e as tecnologias relevantes para a geração de documentos PDF, assinatura digital e envio automatizado de correio eletrónico. No final, é apresentada uma síntese comparativa que evidencia de que forma o sistema desenvolvido se distingue das abordagens existentes.

2.1 Processo de Certificação de Formação em Portugal

O processo de emissão de certificados de formação e de conclusão de cursos em Portugal é regulado por normas estritas, em particular no contexto da formação profissional certificada pela Direção-Geral do Emprego e das Relações de Trabalho (DGERT) [Direção-Geral do Emprego e das Relações de Trabalho (DGERT), 2026] e pelo Sistema de Informação e Gestão da Oferta Educativa e Formativa (SIGO) [ANQEP — Agência Nacional para a Qualificação e o Ensino Profissional, 2026]. A regulamentação exige que as entidades registem e comprovem a frequência e aproveitamento dos formandos através de documentação oficial auditável.

Tipicamente, o fluxo do processo de certificação nas instituições portuguesas compreende as seguintes fases:

1. **Conclusão e Aproveitamento:** O formando conclui a ação de formação com aproveitamento. A coordenação e o formador validam

as avaliações e as presenças em ata.

2. **Registo Oficial:** Os dados do formando, da ação e os resultados são registados na plataforma governamental SIGO, gerando-se um número de registo nacional único para o certificado.
3. **Geração do Documento:** Com base nos dados registados no sistema interno (ERP/LMS) da entidade e no número oficial SIGO, é gerado o layout visual do certificado.
4. **Assinatura e Validação:** O documento é assinado eletronicamente pela coordenação ou direção de formação, recorrendo a chaves criptográficas da instituição de forma a garantir validade legal e integridade física ao documento digital.
5. **Distribuição:** O certificado digital assinado é enviado por correio eletrónico ou disponibilizado em formato digital numa área pessoal de formando.

Um exemplo prático e representativo deste fluxo em Portugal é o adotado pela empresa Rumos [Rumos, 2026], uma das maiores entidades formadoras nacionais em tecnologias de informação. A Rumos processa a conclusão das ações de formação através do seu ERP interno e efetua a geração e a assinatura digitalizada ou criptográfica dos diplomas. Posteriormente, estes certificados digitais são distribuídos automaticamente aos alunos por correio eletrónico ou disponibilizados diretamente na área pessoal do formando para descarga.

O sistema proposto neste relatório visa apoiar e automatizar precisamente as etapas de geração visual, assinatura digitalizada (PKCS#7) e distribuição direta via correio eletrónico, reduzindo a intervenção manual do pessoal administrativo no processamento de volumes elevados de formandos.

2.2 Soluções Comerciais Existentes

Existem atualmente diversas plataformas comerciais direcionadas para a emissão digital de certificados. As mais representativas são descritas a seguir.

A **Accredible** [Accredible, 2024] é uma plataforma *Software as a Service* (SaaS) que permite às organizações criar, gerir e partilhar certificados e *badges* digitais. Suporta o padrão *Open Badges* 2.0, definido pela IMS Global Learning Consortium, o que confere interoperabilidade entre plataformas. Disponibiliza uma API REST para integração com sistemas externos (e.g., plataformas LMS) e incorpora mecanismos de verificação da autenticidade dos certificados emitidos através de URL únicos.

A **Certifier** [Certifier, 2024] é outra plataforma SaaS que disponibiliza um editor visual de templates, automação do envio por correio eletrónico e analítica sobre a receção e partilha dos certificados pelos destinatários. Oferece também integração com plataformas de gestão de eventos e ferramentas de terceiros como o Zapier [Zapier, 2024], que permite a automação de fluxos de trabalho adicionais.

O **Canva** [Canva, 2024], embora não seja uma solução dedicada à emissão de certificados, é amplamente utilizado para a criação visual dos mesmos. Contudo, não oferece qualquer mecanismo de preenchimento automático de dados nem de envio programático, exigindo intervenção manual para cada certificado gerado.

A **Open Badges** [IMS Global Learning Consortium, 2022] é uma especificação aberta desenvolvida pela Mozilla Foundation e posteriormente mantida pela IMS Global Learning Consortium. Define um formato padronizado para credenciais digitais que incorporam metadados verificáveis (e.g., emissor, destinatário, critérios de atribuição). Apesar de não ser uma plataforma por si só, constitui uma referência importante no domínio da certificação digital interoperável.

Importa salientar que todas estas soluções são serviços geridos por terceiros, implicando custos de subscrição recorrentes, dependência de infraestrutura externa e limitações na personalização profunda dos templates e dos fluxos de trabalho. O sistema desenvolvido no âmbito deste projeto surge como uma alternativa *open-source* e auto-hospedada, configurável pelas próprias instituições.

2.3 Geração Programática de Documentos PDF

A geração automática de certificados em formato PDF pode ser abordada através de diferentes paradigmas tecnológicos. Uma primeira abordagem consiste na utilização de bibliotecas orientadas à manipulação direta de PDF, como a **PDFKit** [Grothaus et al., 2024] (para Node.js) ou a **iText** [iText Group NV, 2024] (para Java e .NET). Estas bibliotecas permitem construir os documentos PDF de forma inteiramente programática, definindo explicitamente as posições, as fontes e os conteúdos. Esta via oferece um grande controlo sobre o resultado final, contudo implica uma curva de aprendizagem elevada e dificulta a criação de layouts complexos.

Uma segunda abordagem, adotada no presente projeto, consiste na conversão de documentos HTML/CSS para PDF através de um motor de renderização de navegador (*headless browser*). A biblioteca **Puppeteer** [Google Chrome Team, 2024], desenvolvida pela Google, disponibiliza uma API de alto nível sobre o Chromium que permite, entre outras funcionalidades, exportar páginas web para PDF com elevada fidelidade de renderização. Esta abordagem apresenta vantagens significativas: permite aproveitar toda a expressividade do HTML e do CSS moderno para definir o layout dos certificados, incluindo gradientes, fontes tipográficas personalizadas e posicionamento preciso dos elementos; facilita a separação entre a lógica de negócio e a apresentação; e simplifica a criação de templates por utilizadores sem conhecimentos de programação PDF de baixo nível [Manticore, 2023].

2.4 Assinatura Digital de Documentos

A assinatura digital constitui um mecanismo criptográfico que permite garantir a autenticidade e integridade de um documento eletrónico, associando-o de forma inequívoca ao seu signatário. No contexto dos certificados de curso, a assinatura digital serve para atestar que o documento foi emitido pela entidade competente e que o seu conteúdo não foi alterado após a emissão.

No presente projeto, optou-se pela utilização da biblioteca **node-signpdf** [vbuch and contributors, 2024]. Esta ferramenta permite incorporar assinaturas digitais nos ficheiros PDF gerados pelo Puppeteer, implementando

o padrão PKCS#7. A decisão de utilizar esta biblioteca prendeu-se com dois fatores: em primeiro lugar, a sua integração direta no ecossistema Node.js, dispensando a invocação de ferramentas externas (como o `pdftk` ou o `openssl` via linha de comandos); em segundo lugar, o suporte nativo para certificados no formato `.p12`, que é o formato de chave privada mais comum em contexto institucional. Alternativas como manipular o formato PDF binário diretamente ou utilizar bibliotecas de propósito geral como a `pdf-lib` exigiriam a implementação manual do protocolo PKCS#7, aumentando a complexidade e a superfície de erros. Embora não produza assinaturas qualificadas no rigoroso sentido do regulamento eIDAS, esta abordagem garante com segurança a integridade do documento e a sua correta associação ao certificado digital da instituição emissora, sendo reconhecida por leitores de PDF de referência como o *Adobe Acrobat Reader*.

2.5 Distribuição por Correio Eletrônico Automatizado

O envio automatizado de certificados por correio eletrônico é uma componente central de qualquer sistema de emissão digital. Existem duas abordagens principais: a utilização de servidores SMTP próprios ou de serviços transacionais de terceiros.

A utilização de **SMTP** (*Simple Mail Transfer Protocol*) diretamente, através de bibliotecas como a **Nodemailer** [Reinman, 2024] para Node.js, permite o envio de mensagens com anexos (neste caso, os certificados em PDF) sem dependência de serviços externos, sendo adequada para ambientes auto-hospedados. O Nodemailer suporta autenticação SMTP, TLS/SSL e o envio de conteúdo MIME multipart, o que permite incluir tanto uma versão HTML como uma versão em texto simples da mensagem.

Como alternativa, serviços transacionais como o **SendGrid** [Twilio SendGrid, 2024] oferecem APIs REST de alto nível. Disponibilizam também ferramentas de análise sobre a entrega das mensagens e mecanismos integrados de conformidade com os protocolos DKIM (*DomainKeys Identified Mail*) e SPF (*Sender Policy Framework*). Estas capacidades contribuem decisivamente para melhorar a taxa de entrega e reduzir a probabilidade de as mensagens serem classificadas como *spam*. Em contrapartida, implicam custos

acrescidos e uma forte dependência de infraestruturas externas.

No presente sistema optou-se pela utilização do Nodemailer com um servidor SMTP configurável, mantendo a autonomia da instituição e evitando custos adicionais de subscrição.

2.6 Síntese Comparativa

Apresenta-se na Tabela 2.1 uma síntese comparativa entre as soluções estudadas e o sistema desenvolvido no âmbito deste projeto, tendo em conta os critérios operacionais e económicos mais relevantes para o contexto de utilização pretendido.

Tabela 2.1: Comparação entre soluções de emissão de certificados digitais

Critério	Accredible	Certifier	Canva	Open Badges	Sistema Proposto
Geração automática	✓	✓	×	✓	✓
Templates personalizáveis	✓	✓	✓	×	✓
Assinatura digital	✓	×	×	×	✓
Envio por email	✓	✓	×	×	✓
Auto-hospedado	×	×	×	×	✓
Código aberto	×	×	×	✓	✓
Custo de utilização	Pago	Pago	Misto (Freemium)	Gratuito	Gratuito

Importa clarificar que o termo *Freemium* (uma mistura das palavras em inglês *free* e *premium*) atribuído ao Canva refere-se a um modelo de negócio em que a utilização básica da ferramenta é gratuita, contudo, o acesso a funcionalidades avançadas (como exportação em resoluções superiores, redimensionamento automático de imagens e utilização de determinados elementos gráficos e tipográficos exclusivos) requer a subscrição de um plano pago.

2.6.1 Delimitação do Contexto

O sistema proposto insere-se no domínio da gestão administrativa de secretarias escolares e centros de formação que necessitam de comprovar competências ou participações em atividades através da emissão de diplomas e certificados oficiais. O volume destas emissões e a necessidade de garantir a autenticidade jurídica dos documentos gerados sob fortes restrições de privacidade de dados (em conformidade com o Regulamento Geral sobre a Proteção de Dados - RGPD) e de controlo orçamental delimitam o âmbito deste trabalho.

2.6.2 Aspectos Diferenciadores e Inovadores

O sistema desenvolvido distingue-se das soluções existentes pelos seguintes aspetos de diferenciação prática e inovação operacional:

- **Autonomia e Custo Zero:** Ao contrário de plataformas comerciais como Accredible ou Certifier, que exigem pagamentos recorrentes proporcionais ao volume de certificados emitidos, esta proposta é gratuita e auto-hospedada.
- **Inviolabilidade Local dos Dados:** Sendo executado na própria infraestrutura da instituição, todos os dados sensíveis dos formandos e chaves de assinatura P12 permanecem sob controlo direto do emitente, evitando a transferência de dados para terceiros.
- **Editor Visual com Geração em Lote e Assinatura:** Une a flexibilidade gráfica e simplicidade de desenho de interfaces como o Canva a capacidades de processamento programático assíncrono em lote (lotes de 5 para controle de memória) e assinatura digital PKCS#7 automatizada.

2.6.3 Pressupostos Teóricos e Tecnológicos

A viabilidade técnica do sistema desenvolvido baseia-se nos seguintes pressupostos centrais:

- **Renderização baseada em Motores Web (Headless Browser):** O motor Blink do Chromium (via Puppeteer) constitui a base estável para interpretar layouts HTML5/CSS3 modernos, superando a rigidez e as limitações das bibliotecas clássicas de criação de PDF de baixo nível.
- **Segurança e Validação Criptográfica Assimétrica (PKCS#7):** Assume-se que a autenticidade e integridade física dos documentos são asseguradas por assinaturas baseadas no padrão PKCS#7 utilizando ficheiros de chaves privadas/públicas .p12, que podem ser reconhecidos nativamente por leitores de PDF de mercado (e.g., Adobe Acrobat Reader).

- **Descentralização do Envio de Emails:** A utilização do protocolo SMTP direto (Nodemailer) assume que a instituição possui ou configura servidores de email próprios, evitando custos de subscrição e dependência de APIs transacionais de terceiros.

Capítulo 3

Modelo Proposto

Neste capítulo vamos elaborar sobre o modelo que propusemos para este projeto, começando pelos requisitos levantados, passando por todos os casos de utilização possíveis, os seus fundamentos e no final a abordagem que utilizámos para cada um deles.

3.1 Requisitos

Para a concretização eficiente deste projeto, delineámos de forma rigorosa os objetivos que o sistema deve cumprir. Esta análise baseou-se no levantamento de requisitos funcionais e não-funcionais e na respetiva modelação em casos de utilização.

3.1.1 Atores do Sistema

Foram identificados dois perfis de interação com a plataforma: • **Administrador:** Representa o funcionário administrativo da instituição de ensino. Possui acesso total ao sistema (protegido por credenciais) para gerir utilizadores, configurar as credenciais do correio eletrónico, definir os certificados digitais (.p12), desenhar templates e despoletar os processos de geração e expedição. • **Utilizador / Funcionário:** Perfil com privilégios limitados, cujo acesso cinge-se tipicamente à consulta e descarregamento dos certificados que lhe foram emitidos (caso a instituição opte por disponibilizar o portal aos alunos e não apenas efetuar o envio via email).

3.1.2 Requisitos Funcionais e Não-Funcionais

Os requisitos levantados orientaram as escolhas tecnológicas. Na Tabela (Figura 3.1) indicamos todos os requisitos definidos, onde a parte funcional está classificada como “Função” e a parte não-funcional subjacente está identificada como “Detalhe”.

De forma a sustentar a arquitetura desenvolvida, identificaram-se três pilares de requisitos não-funcionais:

- **Segurança:** Proteção de credenciais críticas através de ficheiros de ambiente e de configuração locais; isolamento de rotas administrativas através de controlo de acessos baseado em papéis (RBAC).
- **Desempenho:** O sistema deve evitar o esgotamento dos recursos locais do servidor quando é solicitada a geração de múltiplos documentos em simultâneo.
- **Usabilidade:** A plataforma deve ocultar a complexidade do motor de geração de documentos através de uma interface web que permita aos funcionários operar o sistema sem formação técnica em programação.

Ref	Função	Categoria	Atributo	Detalhe	Categoria
1	Autenticar o utilizador no acesso ao sistema	Visível	Tolerância a falhas	Garantir a autenticação correta de cada utilizador	Obrigatório
2	Criar utilizadores e atribuir-lhes permissões	Visível	Controlo de Acesso	Apenas um Administrador pode criar utilizadores e atribuir-lhes permissões	Obrigatório
3	Remover um utilizador	Visível	Controlo de Acesso	Apenas um Administrador pode remover um utilizador	Obrigatório
4	Permitir os administradores e funcionários selecionar cursos para gerar certificados	Visível	Tolerância a falhas	Garantir a possibilidade de selecionar todos os cursos	Obrigatório
5	Consultar alunos aprovados num curso	Invisível	Tolerância a falhas	Garantir a obtenção de todos os alunos que concluíram	Obrigatório
6	Obter informação dos alunos	Visível	Tolerância a falhas	Garantir a privacidade da informação	Obrigatório
7	Criar template	Invisível	Tolerância a falhas	Garantir a criação correta de um template de certificado	Obrigatório
8	Editar template	Invisível	Tolerância a falhas	Garantir a edição correta de um template existente	Desejável
9	Eliminar template	Invisível	Tolerância a falhas	Garantir a eliminação segura de um template existente	Desejável
10	Preencher o certificado com a informação do aluno	Invisível	Tolerância a falhas	Garantir a autenticidade da informação	Obrigatório
11	Processar um template para gerar o certificado	Invisível	Tolerância a falhas	Concluir geração de todos os certificados	Obrigatório
			Tempo de Resposta	Concluir geração de 1000 certificados num prazo de 24 horas	Desejável
12	Gerar o certificado	Invisível	Plataforma	Gerar o certificado em formato PDF	Obrigatório
13	Armazenar o certificado	Invisível	Plataforma	Garantir o armazenamento do certificado gerado	Obrigatório
14	Assinar digitalmente o certificado	Visível	Tolerância a falhas	Garantir a autenticidade da assinatura	Desejável
15	Notificar o aluno acerca da disponibilidade do certificado	Visível	Tolerância a falhas	Notificar o aluno apenas após a geração do certificado	Obrigatório
16	Disponibilizar o documento	Visível	Plataforma	Disponibilizar o certificado no email enviado	Obrigatório
17	Eliminar o certificado do armazenamento	Invisível	Plataforma	Garantir a remoção do certificado do armazenamento quando aplicável	Desejável

Figura 3.1: Funções e Atributos do Sistema

3.1.3 Casos de Utilização

Após o levantamento dos requisitos, procedemos à elaboração dos Casos de Utilização (demonstrados na Figura 3.2), que ilustram de forma gráfica as interações entre os atores e as valências do sistema.

Os principais fluxos representados incluem:

1. **Autenticação e Gestão de Permissões:** O processo de autenticação dos utilizadores com verificação das respetivas permissões de acesso.
2. **Gestão de Templates:** O administrador configura os parâmetros visuais e os campos dinâmicos do documento, num formato estruturado e adaptável.
3. **Geração de Certificados e Assinatura:** O administrador aciona a

construção em massa dos diplomas e o sistema aplica, de forma automatizada, a assinatura digital.

4. **Expedição Automática:** O administrador solicita o envio e o sistema distribui os documentos gerados para os respectivos formandos por correio eletrónico.

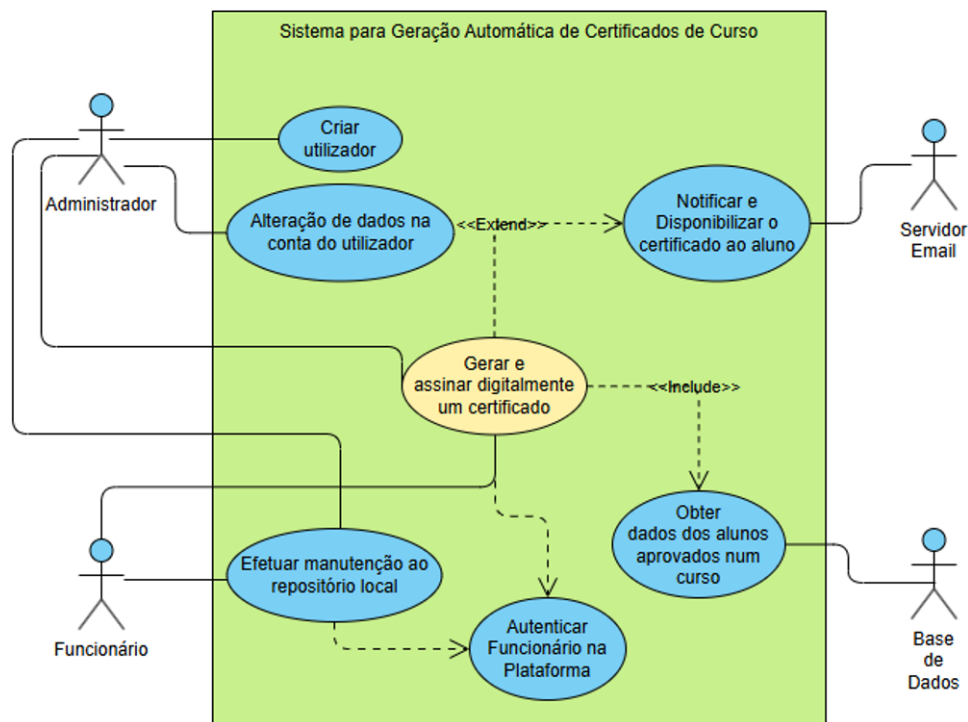


Figura 3.2: Diagrama de Casos de Utilização

De seguida apresentam-se as descrições detalhadas dos casos de utilização principais do sistema.

CU1 — Criar chave de acesso para funcionário

Tabela 3.1: CU1 — Criar chave de acesso para funcionário

Nome	Criar chave de acesso para funcionário
Resumo	Permite ao administrador criar uma chave de acesso para que um funcionário possa aceder ao sistema.
Ator	Administrador
Referências	R1, R2
Fluxo Principal	1. Este caso inicia quando o administrador acede à área de gestão de utilizadores. 2. Seleciona a opção “Criar chave de acesso”. 3. Introduce os dados do funcionário (nome, endereço de correio eletrónico, papel no sistema). 4. O sistema gera uma chave de acesso única. 5. O sistema associa a chave ao funcionário. 6. O sistema apresenta e confirma a chave criada.
Cenário Alt. 1	(Passo 3) Os dados introduzidos são inválidos: o sistema apresenta mensagem de erro e solicita correção; os passos 4, 5 e 6 ficam sem efeito.
Cenário Alt. 2	(Passo 3) O funcionário já possui chave de acesso: o sistema informa e impede a duplicação; os passos 4, 5 e 6 ficam sem efeito.

CU2 — Autenticar funcionário na plataforma

Tabela 3.2: CU2 — Autenticar funcionário na plataforma

Nome	Autenticar funcionário na plataforma
Resumo	Permite ao funcionário autenticar-se no sistema utilizando a sua chave de acesso.
Ator	Funcionário
Referências	R1
Fluxo Principal	1. Este caso inicia quando o funcionário acede à página de autenticação. 2. O funcionário introduz a sua chave de acesso. 3. O sistema valida os dados introduzidos. 4. O sistema autentica o funcionário. 5. O sistema redireciona o utilizador para a interface principal.
Cenário Alt. 1	(Passo 3) A chave de acesso é inválida: o sistema apresenta mensagem de erro; os passos 4 e 5 ficam sem efeito.

CU3 — Obter dados dos alunos aprovados

Tabela 3.3: CU3 — Obter dados dos alunos aprovados

Nome	Obter dados dos alunos aprovados
Resumo	Permite obter a informação dos alunos para verificação e geração de certificados.
Ator	Funcionário / Administrador
Referências	R3, R4, R5
Fluxo Principal	1. Este caso inicia quando o utilizador seleciona a opção de geração de certificados na interface principal. 2. O utilizador seleciona um curso ou define critérios de pesquisa. 3. O sistema consulta a base de dados. 4. O sistema obtém os dados dos alunos aprovados. 5. O sistema apresenta a lista de alunos ao utilizador. 6. O utilizador confirma a veracidade da lista.
Cenário Alt. 1	(Passo 4) Não existem alunos aprovados: o sistema apresenta a lista vazia.
Cenário Alt. 2	(Passo 1) Falta de permissões: acesso negado; os passos 2 a 6 ficam sem efeito.
Cenário Alt. 3	(Passo 6) O utilizador nega a veracidade da lista: o fluxo retorna ao passo 2.

CU4 — Gerar e assinar digitalmente o certificado

Tabela 3.4: CU4 — Gerar e assinar digitalmente o certificado

Nome	Gerar e assinar digitalmente o certificado
Resumo	Permite gerar e assinar digitalmente um certificado de conclusão de curso.
Ator	Funcionário / Administrador
Referências	R6, R7, R8, R9, R10
Fluxo Principal	1. Este caso inicia quando o utilizador confirma o início da geração de certificados. 2. O sistema obtém os dados do aluno. 3. O sistema preenche o template do certificado com os dados do aluno. 4. O sistema converte o certificado para formato PDF. 5. O sistema assina digitalmente o certificado gerado. 6. O sistema armazena o certificado na área de certificados gerados. 7. O sistema confirma a veracidade da assinatura do certificado. 8. O sistema apresenta uma mensagem de conclusão ao utilizador.
Cenário Alt. 1	(Passo 1) O certificado demora demasiado a ser gerado: o sistema apresenta mensagem de erro e solicita nova confirmação; o passo não produz efeito.
Cenário Alt. 2	(Passo 7) O sistema deteta um problema na veracidade do certificado: apresenta mensagem de erro e solicita nova confirmação.

CU5 — Notificar e disponibilizar o certificado ao aluno

Tabela 3.5: CU5 — Notificar e disponibilizar o certificado ao aluno

Nome	Notificar e disponibilizar o certificado gerado ao aluno
Resumo	Permite notificar e disponibilizar o certificado de conclusão ao aluno por correio eletrônico.
Ator	Funcionário / Administrador
Referências	R11, R12
Fluxo Principal	1. Este caso inicia quando o certificado está gerado e o utilizador seleciona a opção de envio na interface principal. 2. O sistema preenche o template da mensagem de correio eletrônico com os dados relevantes. 3. O sistema acrescenta à mensagem uma cópia do certificado. 4. O sistema envia a mensagem completa para o servidor de correio eletrônico e aguarda confirmação.
Cenário Alt. 1	(Passo 4) O servidor de correio eletrônico demora demasiado a confirmar a receção do pedido: o sistema apresenta mensagem de erro e solicita nova confirmação; o envio fica sem efeito.

3.2 Fundamentos

O desenho do sistema baseia-se em padrões de arquitetura de software que pretendem promover a modularidade, a extensibilidade e a segurança da aplicação. De seguida, são detalhados os pilares conceptuais do modelo proposto.

3.2.1 Arquitetura Model-View-Controller (MVC)

Para estruturar o sistema, adotou-se o padrão de desenho de arquitetura MVC, que promove a separação de conceitos ao dividir a aplicação em três

camadas distintas:

- **Model:** Camada que representa as entidades de domínio da aplicação (como utilizadores, alunos, cursos, templates e certificados). Esta camada define as estruturas de dados associados aos elementos do sistema.
- **View:** Responsável pela apresentação da interface visual ao utilizador. Os templates de interface são renderizados no servidor e enviados ao cliente como conteúdo dinâmico.
- **Controller:** Lógica de controlo intermediária que recebe os pedidos do cliente, interage com a camada de serviço e seleciona a vista apropriada. Permite desacoplar a lógica de apresentação da lógica de negócio.

A preferência por MVC face a uma abordagem monolítica (onde a lógica de negócio, o acesso a dados e a apresentação coexistem nas mesmas funções ou ficheiros) justifica-se pela natureza do sistema: este integra simultaneamente um motor de geração de PDFs, um módulo de assinatura digital, um serviço de envio de correio eletrónico e um sistema de controlo de acessos. Sem uma separação explícita de responsabilidades, qualquer alteração a um destes módulos - como a substituição da base de dados ou a adição de uma nova fonte de dados de alunos - poderia afetar os restantes de forma imprevisível. O MVC impõe fronteiras claras que tornam o código mais seguro de modificar e mais fácil de testar em isolamento.

Com vista a reforçar o princípio de responsabilidade única, foram acrescentadas duas camadas adicionais ao padrão MVC clássico. A camada de **Serviço** (*Service*) encapsula a lógica de negócio da aplicação e serve de intermediária entre os controladores e a camada de acesso a dados: é ela que coordena as operações, aplica as regras de negócio e devolve os resultados já tratados ao controlador. A camada de **Acesso a Dados** (*Data Access Object* — DAO) é a única responsável por comunicar diretamente com o mecanismo de persistência, abstraindo-o através de interfaces bem definidas. Esta separação permite, por exemplo, substituir a implementação do armazenamento por uma alternativa em memória para efeitos de teste, sem alterar qualquer outra camada da aplicação.

3.2.2 Motor de Renderização de Documentos PDF

A conversão automatizada de um layout visual num ficheiro PDF é um dos núcleos centrais do projeto. A abordagem adotada baseia-se na utilização de um motor de renderização web (*headless browser*), que interpreta especificações modernas de apresentação visual (como HTML e CSS). Em vez de calcular posições de texto e formas geométricas através de bibliotecas de baixo nível, a aplicação delega a um motor de renderização a composição visual do certificado. Isto permite o uso nativo de tipos de letra, posicionamentos precisos e imagens integradas, produzindo um documento PDF fiel ao layout pretendido.

3.2.3 Segurança e Controlo de Acessos

A proteção dos dados e a validação de privilégios de acesso assentam em duas vertentes:

- **Autenticação e Proteção de Credenciais:** Os dados de autenticação são verificados a partir de derivações criptográficas (*hash*) das palavras-passe armazenadas. A persistência da sessão baseia-se em tokens de sessão seguros.
- **Autorização Baseada em Papéis (RBAC):** O controlo de acessos é implementado segundo o modelo RBAC (*Role-Based Access Control*), que permite definir de forma declarativa as permissões associadas a cada papel de utilizador, garantindo que apenas os administradores possam criar novos utilizadores, manusear o repositório e configurar credenciais para o envio de emails ou para a assinatura digital.

3.3 Abordagem

A abordagem proposta visa eliminar a intervenção manual no processo de produção de certificados, substituindo-a por um conjunto de etapas executadas automaticamente pelo sistema: desde a composição visual do documento, passando pela substituição dos dados de cada aluno, até à sua assinatura e envio por correio eletrónico. A geração dos documentos é realizada de forma não bloqueante, ou seja, o sistema processa os certificados em segundo plano sem impedir a utilização da plataforma enquanto decorre o processamento.

Esta secção detalha a estrutura de representação dos templates e o fluxo de processamento global.

3.3.1 Estruturação e Conceção do Layout de Templates

Para suportar o requisito de personalização por parte das instituições, os templates de certificados não são estáticos. Cada template é guardado no sistema como uma estrutura de dados que descreve o seu layout e os seus elementos constituintes.

O layout armazena propriedades globais do documento, como a cor de fundo e a espessura e cor das bordas do certificado. Adicionalmente, possui uma coleção de elementos dinâmicos e estáticos, onde cada elemento possui propriedades bem definidas (coordenadas, tamanho e família de letra, rotação, cor e alinhamento). Os elementos dividem-se em:

- **Text:** Conteúdo textual fixo inserido pelo criador do template.
- **Placeholder:** Campos dinâmicos mapeados para atributos específicos do aluno ou do curso (como nome, identificador, designação do curso e datas de início e fim), que são substituídos automaticamente pelos dados reais no momento de geração.
- **Image:** Imagens integradas (como logótipos ou fundos decorativos) que complementam a identidade gráfica do certificado.

3.3.2 Algoritmo de Geração de Certificados em Lote

A geração de documentos PDF a partir de uma lista de alunos qualificados, ou seja alunos que acabaram o curso em questão, e de um template selecionado é realizada de forma assíncrona, com o objetivo de manter a estabilidade do sistema sob carga de processamento elevada. O fluxo de execução segue os seguintes passos:

1. **Processamento em Lote (*Batching*):** A lista de alunos é dividida em lotes de dimensão controlada. Isto impede a abertura simultânea de um número excessivo de instâncias de processamento que poderia esgotar os recursos de CPU e memória do servidor.

2. **Substituição de Campos Dinâmicos:** Para cada aluno no lote, os campos dinâmicos (*placeholders*) do template são substituídos pelos dados reais do aluno e do curso, gerando o conteúdo final a posicionar no documento.
3. **Composição do Documento Intermédio:** É gerado um documento de apresentação intermédio que define o layout visual do certificado, com as dimensões proporcionais ao formato A4 em paisagem, posicionando cada elemento de forma absoluta.
4. **Exportação para PDF:** O motor de renderização processa o documento intermédio, aguardando o carregamento de todos os recursos externos (imagens e tipos de letra), e exporta o resultado final para um ficheiro PDF.
5. **Assinatura Digital:** O ficheiro PDF gerado é encaminhado para o módulo de assinatura criptográfica, aplicando o certificado digital da instituição.
6. **Limpeza de Recursos:** Os ficheiros temporários são eliminados e as instâncias de processamento são encerradas, permitindo prosseguir para o lote seguinte.

3.3.3 Fluxo de Distribuição por Correio Eletrónico

Uma vez gerado o ficheiro PDF, o certificado fica disponível no repositório do sistema e é identificado de forma unívoca com base no aluno e no curso a que pertence. A sua distribuição é gerida pelo módulo de correio eletrónico:

- O utilizador seleciona os alunos para os quais deverá ser enviado o certificado.
- O sistema verifica se o ficheiro PDF correspondente ao certificado existe localmente. Caso não exista, a operação é registada como ignorada para o aluno em causa.
- O sistema estabelece uma ligação ao servidor de correio eletrónico configurado pela instituição, gera uma mensagem com conteúdo personalizado e anexa o certificado em PDF gerado.

O fluxo garante que todo o percurso, desde o desenho do template à distribuição dos certificados assinados, é transparente e auditável pela instituição.

Capítulo 4

Implementação do Modelo

Este capítulo descreve, com detalhe técnico, como os requisitos definidos no capítulo 3 foram implementados, evidenciando as escolhas tecnológicas e a integração entre os componentes.

4.1 Arquitetura Geral do Sistema

O sistema foi desenvolvido seguindo o padrão de arquitetura **Model-View-Controller (MVC)** em conjunto com o padrão de acesso a dados **Data Access Object (DAO)**. Esta escolha permite separar claramente a lógica de negócios, a camada de apresentação e o acesso persistente, facilitando a manutenção e a extensibilidade.

Para o servidor web, foi adotado o *framework* **Express.js**. A preferência por Express em detrimento de alternativas como o Fastify ou o Koa justifica-se pela maturidade da sua plataforma de *middleware* e pela ampla adoção no ecossistema Node.js, o que se traduz numa grande disponibilidade de bibliotecas compatíveis (como o Casbin, o Multer e o express-session) já testadas em conjunto com o framework. Para a camada de apresentação, optou-se por **EJS** (*Embedded JavaScript*), um motor de templates que integra lógica de servidor diretamente em HTML. A escolha recaiu sobre EJS face a alternativas como o Pug ou o Handlebars porque o EJS preserva a sintaxe HTML nativa — o que facilita a adaptação por parte de qualquer elemento da equipa com conhecimentos base de HTML — e permite embutir expressões JavaScript sem impor um novo sistema de sintaxe a aprender.

A Figura 4.1 ilustra a organização em camadas do sistema, evidenciando o

fluxo de dados desde o pedido HTTP do cliente até à camada de persistência.

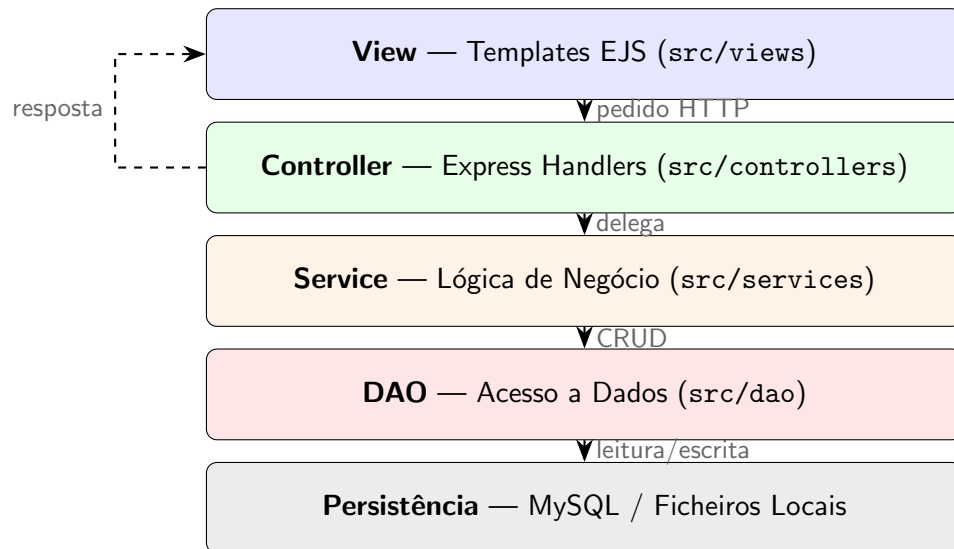


Figura 4.1: Arquitetura em camadas do sistema (MVC + Service + DAO)

4.1.1 Camada de Modelo

Os objetos de domínio são representados por classes TypeScript situadas em `src/model`, incluindo `Student`, `Course`, `Template` e `Certificate`. Cada entidade possui um interface DAO correspondente (por exemplo, `IStudentDAO`) que define as operações CRUD. As implementações concretas estão em `src/dao/implementations` e podem ser alternadas entre armazenamento em ficheiro local e base de dados MySQL, permitindo fácil migração de ambientes.

Esta arquitetura traz uma vantagem estratégica relevante: para ligar o sistema a uma nova fonte de dados (por exemplo, uma base de dados académica proprietária de outra instituição), basta criar uma nova classe que implemente a interface DAO correspondente, **sem necessidade de alterar qualquer outra camada do sistema**. Isto confere ao sistema uma elevada portabilidade institucional, tornando-o facilmente adotável por diferentes instituições de ensino, independentemente da tecnologia de armazenamento de dados que utilizem.

4.1.2 Camada de Controlo

Os controladores Express (`src/controllers`) recebem as requisições HTTP, delegam à camada de serviço e retornam as *views*. Exemplos críticos incluem `certificates.ts` que orquestra a geração de PDFs, assinatura digital e envio de emails, e `settings.ts` que gerencia a configuração segura das credenciais SMTP e da chave de assinatura.

4.1.3 Camada de Visualização

A apresentação utiliza templates EJS (`src/views`) renderizados no servidor. Todos os estilos foram extraídos para o ficheiro global `public/css/main.css`, seguindo boas práticas de separação de responsabilidades e permitindo a aplicação de um design consistente e responsivo.

4.1.4 Diagrama de Componentes do Sistema

A organização física e as dependências entre os módulos de *software* da aplicação são ilustradas na Figura 4.2 através de um diagrama de componentes UML. Este diagrama evidencia a separação clara entre os serviços centrais (geração, assinatura e envio) e os recursos de suporte externos, demonstrando como a camada de controlo orquestra as operações.

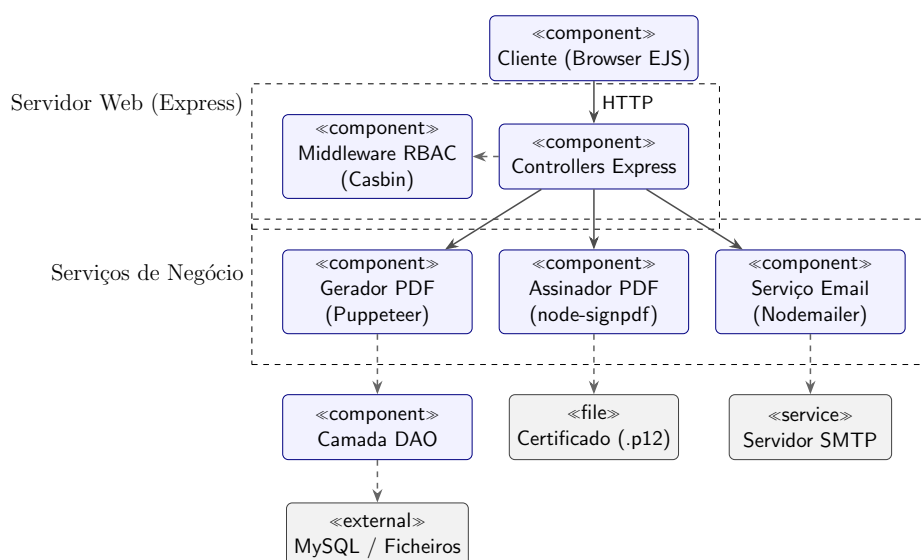


Figura 4.2: Diagrama de componentes do sistema

4.2 Geração de Certificados PDF

A geração dos certificados baseia-se na biblioteca **Puppeteer**. O serviço `src/services/certificate/generator.ts` converte, em lote, um *layout* JSON (`TemplateLayout`) em HTML, abre uma página *headless* e exporta o resultado para PDF com as dimensões exatas do certificado (1485px × 1050px). O algoritmo processa os alunos em lotes de cinco para otimizar o uso de memória e CPU.

Código 1: Fluxo de geração de PDF

```
async function generatePdfCertificates(
  students: Student[],
  layout: TemplateLayout
) {
  // prepares directories
  fs.mkdirSync(CERTIFICATE_REPOSITORY_DIR, {recursive:true});
  fs.mkdirSync(path.dirname(tempCertificatePath), {recursive:true});

  const browser = await puppeteer.launch({
    headless:true,
    args:['--no-sandbox','--disable-setuid-sandbox','--disable-dev-shm-usage']
  });
  const batchSize = 5;
  for (let i = 0; i < students.length; i += batchSize) {
    const batch = students.slice(i, i + batchSize);
    await Promise.all(batch.map(async student => {
      const page = await browser.newPage();
      try {
        await generateTempHTML(layout, student);
        await generatePDF(student, page);
        await signCertificate(student);
      } catch (err) {
        console.error('Failed to generate for ${student.name}:', err);
      } finally {
        fs.unlink(tempCertificatePath, (err) => { });
        await page.close();
      }
    }));
  }
  await browser.close();
}
```

4.3 Assinatura Digital

A assinatura emprega **node-signpdf** com um certificado **.p12** configurado pelo administrador na página de definições. O módulo **signCertificate** em **src/services/certificate/generator.ts** insere um *placeholder* no ficheiro temporário, carrega as definições da assinatura e a respetiva palavra-passe do certificado, e gera a assinatura no formato PKCS#7. A palavra-passe do certificado é guardada em repouso no ficheiro de configuração local, sendo carregada pelo servidor no momento da assinatura para inicializar o assinador, garantindo que o processo ocorre de forma automática.

4.4 Envio Automático de Emails

O envio de correio eletrónico utiliza o **Nodemailer**¹ com um transporte SMTP parametrizado a partir de variáveis de ambiente. As credenciais e configurações necessárias (host, utilizador, porta e palavra-passe) são definidas no ficheiro **.env** e importadas pelo módulo **src/configs/email.ts**. O serviço **sendUserCertificateEmail** no módulo **sendEmail.ts** inicializa o remetente (**createTransporter**) e lança uma exceção caso as configurações necessárias não estejam presentes, permitindo ao sistema notificar o administrador sobre falhas no envio.

Código 2: Fluxo de geração de PDF

```
export const sendUserCertificateEmail = async (
  student: Student,
  filePath?: string
): Promise<void> => {
  const certificate = await dao.getCertificateByStudent(student);
  if (certificate == null) { return; }

  // lana erro se faltarem credenciais no .env
  const transporter = createTransporter();
  const mailOptions: SendMailOptions = { /* ... */ };

  // Erros de envio so propagados para o controlador superior
  await transporter.sendMail(mailOptions);
  console.log('Email sent to ${student.email}!');
};
```

¹O Nodemailer é um módulo do ecossistema Node.js que permite a expedição de mensagens de correio eletrónico a partir de aplicações JavaScript, com suporte para o protocolo SMTP e tratamento de anexos.

4.5 Segurança e Proteção de Dados

A segurança e a proteção de dados na aplicação assentam em diferentes mecanismos no servidor:

- **Cifragem de Palavras-passe:** O armazenamento das credenciais dos utilizadores não é feito em texto limpo; em vez disso, a aplicação utiliza a biblioteca `bcrypt-ts` para gerar *hashes* seguros e com *salt* das palavras-passe. A opção pelo algoritmo `bcrypt` face a alternativas como SHA-256 ou MD5 justifica-se pelo facto de o `bcrypt` ser deliberadamente computacionalmente custoso (através de um fator de custo configurável), o que torna impraticáveis os ataques de *brute-force* e de dicionário, mesmo que a base de dados de credenciais seja comprometida.
- **Segurança de Sessões:** O estado de autenticação e o papel do utilizador são geridos através de *cookies* de sessão seguros no navegador, configurados com os atributos `httpOnly` (impedindo o acesso via scripts do lado do cliente) e `sameSite='lax'` (mitigando ataques de CSRF).
- **Isolamento de Credenciais:** As definições e credenciais de acesso ao servidor SMTP para o envio de correio eletrónico residem unicamente no ficheiro de variáveis de ambiente (`.env`), garantindo que dados confidenciais não são expostos no repositório de código fonte.
- **Assinatura Criptográfica:** O certificado digital `.p12` e as suas definições são acedidos diretamente pelo servidor de forma local, permitindo aplicar assinaturas PKCS#7 aos ficheiros PDF gerados, garantindo a integridade dos certificados emitidos.

4.6 Controlo de Acessos baseado em RBAC

A biblioteca **Casbin** [Luo et al., 2024] define duas políticas principais: *admin* (acesso total para tarefas de configuração, manutenção e gestão de utilizadores) e *user* (acesso a tarefas operacionais de gestão de templates e geração/envio de certificados). As regras estão em `src/configs/casbin/policies/policy.csv` e são carregadas em

`src/middlewares/authorize.ts`. O *middleware* garante que apenas os utilizadores com o papel de administrador (*admin*) podem aceder às rotas críticas de administração e manutenção (`/admin/...`), enquanto as operações comuns de geração e envio de certificados permanecem disponíveis para ambos os perfis.

A decisão de usar o Casbin em vez de uma implementação RBAC à medida assenta na sua abordagem declarativa: as regras de autorização são definidas num ficheiro de política externo (`policy.csv`), sem necessidade de dispersar verificações de permissões pelo código dos controladores. Isto torna o sistema de controlo de acessos simples de entender (basta consultar um único ficheiro para perceber quem pode fazer o quê) e facilmente extensível (adicionar um novo papel ou permissão requer apenas uma nova linha no ficheiro de política, sem alterar código de aplicação).

4.7 Fluxo de Trabalho Integrado

Este fluxo descreve a sequência de passos realizados pelo administrador e pelo sistema para a emissão e distribuição dos certificados. A Figura 4.3 apresenta o diagrama de atividade correspondente a este cenário principal de utilização (o *workflow* geral), sendo os passos detalhados em seguida:

1. O administrador fornece as configurações de email via ambiente (`.env`) e o certificado digital na página de *Definições*.
2. Seleciona um template e filtra os estudantes elegíveis.
3. Aciona a operação para *Gerar certificados*, que procede à criação de PDFs em lote e, opcionalmente, os assina digitalmente.
4. Opcionalmente, invoca o envio de emails contendo o certificado em anexo para cada estudante.
5. O sistema regista a atividade e fornece um resumo dos sucessos e falhas, notificando o utilizador em conformidade nas views correspondentes.

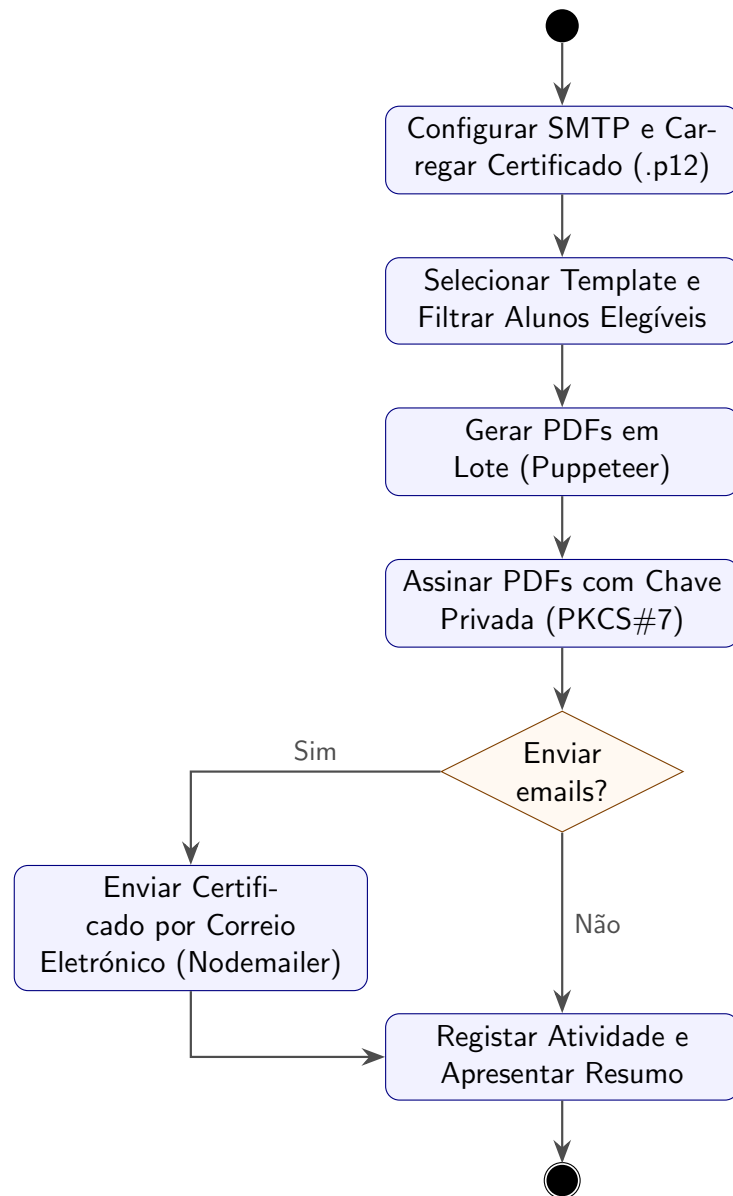


Figura 4.3: Diagrama de atividade do cenário principal (workflow geral)

Capítulo 5

Validação e Testes

Validação e testes são etapas para garantir a robustez e a fiabilidade do Sistema para Geração Automática de Certificados de Curso. A validação assegurou que o sistema cumpre com os requisitos descritos no capítulo 3, englobando as vertentes de geração de ficheiros, autenticidade, envio de mensagens e controlo de acessos. Como a aplicação interage diretamente com ficheiros de grande dimensão e serviços externos, as metodologias de teste focaram-se essencialmente na verificação funcional e no comportamento do sistema sob condições de integração.

5.1 Validação da Geração de Certificados em Lote

A geração de certificados envolve a transformação de um *layout* JSON, passando por um ficheiro temporário HTML, até culminar num ficheiro PDF utilizando a biblioteca Puppeteer. A validação deste processo obedeceu a dois critérios principais:

- **Fidelidade Visual:** Verificou-se se os estilos CSS e o posicionamento absoluto definidos no *template* refletem-se perfeitamente no PDF gerado, preservando as resoluções de 1485 px × 1050 px.
- **Gestão de Recursos (Lotes):** O sistema foi submetido a testes processando dezenas de estudantes simultaneamente. Foi confirmada a eficácia do processamento segmentado em lotes de cinco (*batchSize* = 5). Através de monitorização de sistema, observou-se que o consumo

de memória RAM manteve-se estável, prevenindo falhas por exaustão (*Out Of Memory*) que ocorriam quando se abriam dezenas de instâncias ou separadores do Puppeteer em simultâneo.

5.2 Validação da Assinatura Digital

Para que o sistema possua validade institucional, a assinatura digital dos certificados PDF precisa de cumprir com a norma PKCS#7. O processo de validação da assinatura digital consistiu nas seguintes etapas:

1. **Verificação da Frase-passe e Criptografia:** Validou-se que a configuração da palavra-passe do certificado P12 é corretamente gerida via painel de administração e que falhas na inserção (palavra-passe incorreta ou ausência do certificado) são registadas, impedindo a geração de PDFs mal formatados.
2. **Verificação Externa via Leitores PDF:** Os certificados gerados e assinados foram abertos no *Adobe Acrobat Reader*. O *software* validou automaticamente a assinatura digital, exibindo a barra verde indicadora de integridade. Confirmou-se que o *placeholder* estava adequadamente preenchido e que qualquer tentativa posterior de adulteração do PDF resultou num aviso de que o ficheiro sofreu modificações após ter sido assinado.

5.3 Testes de Integração com o Serviço de Email

O envio de emails através do *Nodemailer* depende da parametrização correta das variáveis de ambiente (via *.env*). Os testes nesta componente incidiram no tratamento e isolamento de falhas, para garantir que o utilizador obtém o devido *feedback*:

- **Teste de Conectividade Bem-Sucedida:** Configurando o servidor SMTP válido, verificou-se o envio da mensagem com o certificado PDF corretamente anexado, e a receção do mesmo na caixa de correio do destinatário sem constrangimentos de *spam*.

- **Tratamento de Exceções:** Foram induzidos erros propositados (omissão de variáveis no `.env`, porta SMTP incorreta e palavras-passe inválidas). O sistema comportou-se de forma resiliente: em vez de a aplicação Node.js terminar inesperadamente, o módulo `sendUserCertificateEmail` propagou o erro para o controlador. Este capturou a exceção e exibiu uma mensagem de alerta detalhada (e.g., “Email não configurado”) diretamente na interface (Dashboard), informando o administrador sobre o estado de falha na notificação de determinados alunos.

5.4 Validação do Controlo de Acessos (RBAC)

O sistema de Controlo de Acessos Baseado em Funções (RBAC), impulsionado pela biblioteca *Casbin*, constitui a linha da frente na segurança do sistema. Foram conduzidos os seguintes cenários de validação funcional:

Perfil / Rota	/ (Dashboard)	/templates (Editor)	/admin/settings (Definições)
Utilizador Regular (<i>user</i>)	Permitido	Permitido	Negado (Redirecionado)
Administrador (<i>admin</i>)	Permitido	Permitido	Permitido

Tabela 5.1: Matriz de validação de acessos (RBAC)

A matriz evidencia que o *middleware* restringe efetivamente o acesso de perfis regulares, protegendo de forma cabal as definições de certificados, assinaturas e templates.

5.5 Conclusão da Validação

Os resultados obtidos através da validação permitiram consolidar a fiabilidade do modelo implementado. A combinação de uma correta gestão de recursos do Puppeteer, as salvaguardas robustas contra falhas no envio SMTP e a inviolabilidade atestada da assinatura digital garantem que a solução está pronta para responder às exigências e à escalabilidade de uma instituição educativa.

Capítulo 6

Conclusões e Trabalho Futuro

Este capítulo encerra o relatório apresentando as principais conclusões retiradas do desenvolvimento do projeto, os resultados obtidos face aos objetivos definidos e um conjunto de propostas de trabalho futuro para a expansão e melhoria da plataforma.

6.1 Conclusões

A concretização deste projeto culminou no desenvolvimento de um *Sistema para Geração Automática de Certificados de Curso* que cumpre os requisitos propostos, desenhado para aliviar a carga administrativa de instituições de ensino. Durante o percurso de implementação, demonstrou-se que a automação de fluxos de trabalho recorrentes - como a criação visual de certificados, a atribuição de validade legal e a sua expedição - não apenas reduz drasticamente os erros operacionais, como também fomenta a escalabilidade.

Entre os principais marcos alcançados, destacam-se:

- A adoção da arquitetura **MVC** (Model-View-Controller) com a linguagem **TypeScript**. Esta combinação revelou-se superior a outras abordagens de desenvolvimento: a arquitetura MVC permitiu desacoplar a lógica de negócio (camada de serviço), a persistência de dados (DAO) e a apresentação (EJS), facilitando a manutenção do código e permitindo alternar de base de dados (MySQL/ficheiros locais) sem impactos noutras camadas, ao passo que uma estrutura homogénea sem divisões claras de responsabilidades complicaria a evolução do sistema.

A escolha de TypeScript face ao JavaScript ajudou na deteção de erros na manipulação de estruturas complexas (como *layouts* e definições SMTP) em tempo de compilação, o que assegurou o correto funcionamento do *software*.

- A geração fiável de ficheiros PDF através do **Puppeteer**, gerida em lotes para otimizar os recursos do servidor e evitar disrupções operacionais (*Out Of Memory*).
- A aplicação integral da **Assinatura Digital PKCS#7**, um passo crucial que autentica os documentos emitidos perante a comunidade e o mercado de trabalho, garantindo a sua integridade.
- A expedição automatizada utilizando **Nodemailer**, suportada por uma arquitetura onde as falhas nas credenciais não provocam colapsos no sistema, garantindo um sistema resiliente.
- O controlo de acesso com o mecanismo **RBAC (Casbin)**, permitindo um distanciamento seguro entre utilizadores regulares e o perfil de administração.
- A **extensibilidade a múltiplas fontes de dados**, proporcionada pela camada DAO. O sistema foi concebido de forma a que a ligação a qualquer base de dados de alunos - seja uma base de dados SQL, um serviço externo ou um ficheiro local - se efetua quase sem alteração ao código existente: basta implementar a interface DAO correspondente. Esta característica torna o sistema facilmente adotável por diferentes instituições de ensino, independentemente da sua infraestrutura de dados.
- A **criação de um editor de templates nativo**. Em vez de depender de ferramentas proprietárias externas (como o Adobe InDesign ou o Microsoft Publisher), o sistema integra um editor visual próprio para a criação de modelos de certificados. Esta decisão traz duas vantagens fundamentais: por um lado, elimina qualquer dependência de licenças comerciais ou de software de terceiros; por outro, permite que a equipa de desenvolvimento controle integralmente o nível de complexidade da

edição disponível, adaptando-o às necessidades reais dos utilizadores da plataforma.

- **A gestão automática do espaço em disco** ocupado pelos certificados gerados. O sistema controla automaticamente os ficheiros PDF armazenados localmente, eliminando certificados desnecessários ou substituídos, reduzindo significativamente a necessidade de manutenção periódica por parte dos administradores e prevenindo o crescimento descontrolado do armazenamento ao longo do tempo.

O modelo proposto provou, assim, responder afirmativamente aos objetivos iniciais. A fusão das mais recentes bibliotecas Node.js demonstrou que é exequível substituir o moroso trabalho manual por uma plataforma centralizada e inteligente.

6.2 Trabalho Futuro

Embora a plataforma se encontre num estado funcional, existe margem para evoluções futuras. Tendo em conta que o sistema foi desenhado para ser integrado de forma local e isolada nos servidores de cada instituição de ensino, a experiência adquirida revelou linhas de melhoria contínua que poderão vir a compor futuras iterações:

1. **Integração com Sistemas Académicos (APIs):** Uma vez que o sistema opera no ecossistema local da instituição, poderá ser desenvolvida uma camada de *Webhooks* ou APIs RESTful que permita a comunicação direta com os sistemas de gestão académica (por exemplo, Moodle, Fénix, etc.). Isto permitiria a emissão dos certificados de forma totalmente automatizada no exato momento em que as pautas finais são lançadas, dispensando intervenção humana.
2. **Módulo de Importação em Massa:** Para as instituições onde não seja viável uma integração via API, a introdução de uma capacidade de *upload* de pautas de avaliação ou folhas de cálculo (CSV / Excel) permitiria inserir automaticamente turmas inteiras no sistema de uma só vez.

3. **Internacionalização (i18n):** Dotar a plataforma de suporte multilíngue, permitindo não apenas a tradução da interface de administração, mas sobretudo a emissão de certificados e o envio de emails em diferentes idiomas, dependendo da origem dos estudantes a certificar.
4. **Dashboard Analítico Avançado:** O desenvolvimento de métricas mais complexas na página inicial, permitindo consultar dados de utilização de recursos da instituição, relatórios anuais de emissão de certificados e validação da taxa de sucesso da entrega de emails.
5. **Gestão de Utilizadores mais segura:** Melhorar a lógica de administração de utilizadores para garantir que um administrador só pode apagar outro administrador se ainda permanecerem outros ativos no sistema. Esta regra evita que o último administrador seja removido, preservando sempre um acesso administrativo válido e evitando bloqueios de gestão.

Em suma, este projeto constitui um pilar fundamental para a desmaterialização burocrática, funcionando como uma base técnica viva e devidamente estruturada, sobre a qual futuros intervenientes poderão escalar e adicionar novas dimensões funcionais.

Apêndice A

Gestão e Controlo de Versões

A gestão de versões do projeto foi efetuada recorrendo ao sistema de controlo de versões **Git**, com repositório remoto alojado na plataforma **GitHub**. Esta arquitetura permitiu salvaguardar o progresso do desenvolvimento, manter um histórico linear de todas as alterações e possibilitar a reversão segura de código em caso de anomalias inesperadas.

A estratégia de ramificação (*branching*) adotada seguiu um fluxo de trabalho focado em isolar as funcionalidades em desenvolvimento do código estável:

- **main:** O ramo principal e estável do repositório. Este ramo reflete apenas o código finalizado, devidamente validado e pronto para implantação na infraestrutura da instituição de ensino.
- **Feature Branches:** Para o desenvolvimento de funcionalidades específicas e isoladas (tais como a integração com a biblioteca *Puppeteer*, a implementação do *Nodemailer* ou a configuração de permissões com *Casbin*), utilizaram-se ramos paralelos. Uma vez concluída e verificada a nova funcionalidade, procedeu-se à junção (*merge*) das alterações para o ramo principal de forma controlada.

O repositório do GitHub serviu não apenas como uma plataforma central de sincronização de código, mas também como base para a documentação primária (*README*) da instalação do projeto.

Apêndice B

Estrutura de Layout JSON dos Certificados

A geração dinâmica dos certificados é assegurada pela leitura de um layout parametrizado no formato JSON, o qual dita as regras de posicionamento, estilização e hierarquia de cada elemento visual no documento final (PDF). Este apêndice detalha a estrutura base utilizada pela camada de modelo (`TemplateLayout`).

O ficheiro JSON de um *template* divide-se em dois blocos organizacionais principais:

1. **page:** Contém as definições globais da folha, nomeadamente as cores de fundo (`backgroundColor`) e cor/espessura da margem limítrofe (`borderColor`). A área útil é gerada automaticamente para a dimensão nativa estipulada de 1485 px × 1050 px.
2. **elements:** Um *array* (*lista*) que engloba todos os blocos de texto, logótipos e contentores de assinatura. Cada elemento declara propriedades CSS personalizadas (`style`), marcação de conteúdo (`content`) e metadados (*flags* booleanas como `isImage` e `isSignature`). Durante o processo de compilação, o sistema substitui automaticamente as variáveis englobadas por chavetas duplas (e.g., `{{name}}`) pela informação real do formando presente no sistema.

Abaixo apresenta-se um excerto ilustrativo de uma configuração JSON correspondente a um certificado padrão:

Código 3: Excerto do layout de um Template em notação JSON

```

{
  "name": "Certificado de Formacao",
  "layout": {
    "page": {
      "backgroundColor": "#ffffff",
      "borderColor": "#0b3169",
      "borderWidth": 15
    },
    "elements": [
      {
        "content": "{{name}}",
        "style": "top: 400px; font-size: 48px; font-weight: bold; color: #333;",
        "isCentered": true
      },
      {
        "content": "Concluiu com sucesso o curso de {{curso}}.",
        "style": "top: 500px; font-size: 24px; color: #666;",
        "isCentered": true
      },
      {
        "content": "Assinatura da Direcao",
        "style": "top: 750px; left: 800px; width: 300px; font-size: 18px;",
        "isSignature": true,
        "isCentered": false
      }
    ]
  }
}

```

Esta estrutura confere uma enorme flexibilidade arquitetural ao sistema, permitindo a instituição de ensino a readaptar a formatação gráfica dos diplomas unicamente alterando o dicionário JSON na base de dados, sem qualquer necessidade de editar o código-fonte (*backend*) da plataforma.

Apêndice C

Ficheiros com etiqueta #my_code

Esta secção lista os ficheiros de código onde foi adicionada explicitamente a etiqueta `my_code` para indicar a nossa contribuição individual.

- `Code/src/controllers/admin.ts`
- `Code/src/controllers/auth.ts`
- `Code/src/controllers/certificates.ts`
- `Code/src/controllers/dashboard.ts`
- `Code/src/controllers/maintenance.ts`
- `Code/src/controllers/settings.ts`
- `Code/src/controllers/templates.ts`
- `Code/src/dao/implementations/local/certificateDAO.ts`
- `Code/src/dao/interfaces/ICourseDAO.ts`
- `Code/src/dao/interfaces/IStudentDAO.ts`
- `Code/src/dao/interfaces/ITemplateDAO.ts`
- `Code/src/dao/interfaces/IUserDAO.ts`
- `Code/src/model/course.ts`
- `Code/src/model/student.ts`

- `Code/src/model/user.ts`
- `Code/src/routes/auth.ts`
- `Code/src/routes/certificates.ts`
- `Code/src/routes/index.ts`
- `Code/src/routes/maintenance.ts`
- `Code/src/routes/settings.ts`
- `Code/src/routes/templates.ts`
- `Code/src/services/auth/auth.ts`
- `Code/src/services/certificate/certificates.ts`
- `Code/src/services/certificate/generator.ts`
- `Code/src/services/certificate/sendEmail.ts`
- `Code/src/services/certificate/template.ts`
- `Code/src/services/maintenance/maintenance.ts`

A marcação `my_code` foi usada para assinalar bocados de código com maior contributo individual, em especial a lógica de geração/envio de certificados, a manutenção dos ficheiros gerados, os controladores e as rotas principais.

Bibliografia

[Accredible, 2024] Accredible (2024). Accredible — digital credentials platform. <https://www.accreditable.com/>. Acedido em: junho de 2025.

[ANQEP — Agência Nacional para a Qualificação e o Ensino Profissional, 2026] ANQEP — Agência Nacional para a Qualificação e o Ensino Profissional (2026). Sistema de informação e gestão da oferta educativa e formativa (sigo). <https://www.dgeec.mec.pt/>. Acedido em: julho de 2026.

[Canva, 2024] Canva (2024). Canva — online design and visual communication platform. <https://www.canva.com/>. Acedido em: junho de 2025.

[Certifier, 2024] Certifier (2024). Certifier — digital certificate platform. <https://certifier.io/>. Acedido em: junho de 2025.

[Direção-Geral do Emprego e das Relações de Trabalho (DGERT), 2026] Direção-Geral do Emprego e das Relações de Trabalho (DGERT) (2026). Certificação de entidades formadoras. <https://www.dgert.gov.pt/>. Acedido em: julho de 2026.

[Google Chrome Team, 2024] Google Chrome Team (2024). Puppeteer — headless chrome node.js api. <https://pptr.dev/>. Acedido em: junho de 2025.

[Grothaus et al., 2024] Grothaus, D. et al. (2024). PDFKit — a javascript pdf generation library for node.js. <https://pdfkit.org/>. Acedido em: junho de 2025.

[IMS Global Learning Consortium, 2022] IMS Global Learning Consortium (2022). Open badges specification, version 2.1. <https://www.imsglobal.org/spec/ob/v2p1/>. Acedido em: junho de 2025.

- [iText Group NV, 2024] iText Group NV (2024). itext — pdf library for java and .net. <https://itextpdf.com/>. Acedido em: junho de 2025.
- [Luo et al., 2024] Luo, Y. et al. (2024). Casbin — authorization library for node.js, goLang, and other languages. <https://casbin.org/>. Acedido em: junho de 2025.
- [Manticore, 2023] Manticore (2023). Html to pdf conversion: Puppeteer vs. wkhtmltopdf. <https://manticore.law/blog/html-to-pdf-puppeteer-vs-wkhtmltopdf>. Acedido em: junho de 2025.
- [Reinman, 2024] Reinman, A. (2024). Nodemailer — send emails from node.js. <https://nodemailer.com/>. Acedido em: junho de 2025.
- [Rumos, 2026] Rumos (2026). Rumos — formação e consultoria em tecnologias de informação. <https://www.rumos.pt/>. Acedido em: julho de 2026.
- [Twilio SendGrid, 2024] Twilio SendGrid (2024). Sendgrid — email delivery service. <https://sendgrid.com/>. Acedido em: junho de 2025.
- [vbuch and contributors, 2024] vbuch and contributors (2024). node-signpdf — a simple pdf signer for node.js. <https://github.com/vbuch/node-signpdf>. Acedido em: junho de 2025.
- [Zapier, 2024] Zapier (2024). Zapier — automation that moves you forward. <https://zapier.com/>. Acedido em: junho de 2025.